

S³-R1: Learning to Retrieve and Answer Step-by-Step with Synthetic Data

Harsh Goel*

The University of Texas at Austin

Akhil Udathu

Google DeepMind

Susmija Jabireddy

Google DeepMind

Pradnesh Kalkar

Google DeepMind

Atharva Parulekar

Google DeepMind

Abstract

Reinforcement learning (RL) post-training has enabled newer capabilities in models, such as agentic tool-use for search. However, these models struggle primarily due to limitations with sparse outcome-based rewards and a lack of training data that encapsulates questions of differing hardness, which results in models not performing deeper searches with tools to collect evidence for question-answering. To address these limitations, we introduce S³-R1 (Synthetic data and stabilized Search R1), a framework that couples a data-centric approach with denser learning signals. We first develop a synthetic generation and curation pipeline that programmatically derives diverse, multi-hop questions from existing documents. This pipeline incorporates a retrieval-based verification step to specifically isolate questions of intermediate difficulty. We then pair this expanded training set with a reward structure that evaluates both intermediate search quality and the correctness of the final answer. This setup directly mitigates the credit assignment problems inherent to sparse rewards. Our evaluations show that S³-R1 outperforms existing baselines by learning more effective search and synthesis strategies, yielding up to a 10% improvement in robust generalization on out-of-domain datasets.

1 Introduction

Large Language Models (LLMs) have demonstrated remarkable capabilities across multiple domains, driving a rapid transition toward an agentic paradigm where models act as autonomous problem-solvers (Team et al., 2024; Anthropic, 2024; OpenAI, 2024). However, their reliance on static, parametric knowledge limits their effectiveness on tasks requiring access to real-time information, specifically in Question-Answering (QA). While Retrieval-Augmented Generation (RAG) (Lewis et al., 2020) alleviates this limitation, contemporary LLMs still heavily struggle with complex, multi-hop questions that require iterative reasoning and strategic tool-use to retrieve the correct content (Trivedi et al., 2022a;b).

To orchestrate this sequential tool-use, recent methods frame multi-hop QA as an interactive process, utilizing external search engines to construct reasoning trajectories (Wu et al., 2024b; Jin et al., 2025b; Song et al., 2025). Consequently, Reinforcement Learning (RL) has emerged as a powerful paradigm for training these agentic policies (Guo et al., 2025a). Frameworks like Search-R1 (Jin et al., 2025b) and R1-Searcher (Song et al., 2025) successfully apply RL to teach models to autonomously generate search queries, process retrieved results, and perform step-by-step reasoning. These methods typically optimize the model using outcome-based rewards derived solely from the final answer’s correctness.

While RL-based post-training has improved QA, gains are increasingly constrained by the training distribution and sparse, outcome-only rewards. If training data rarely requires

*Work done during an internship at Google DeepMind.

multi-hop reasoning and rewards score only the final answer, RL overfits to short-horizon patterns and fails to generalize across hops. Existing synthetic QA pipelines mostly generate reasoning traces for a fixed set of questions (Goldie et al., 2025), which helps credit assignment but does little to broaden the question distribution itself. By contrast, self-play and synthetic generation in formal domains (e.g., math and programming) explicitly expand problem difficulty and diversity (Chen et al., 2025; Zeng et al., 2025).

Inspired by these formal domain approaches that systematically expand the training distribution across varying difficulty levels, we introduce S^3 -R1 that first aims to construct a similar paradigm for complex, multi-hop QA. Our **key technical insight** is to synthesize verified yet solvable question instances by utilizing a frontier model to mutate problems that the base agent fails to solve. To ensure these questions are viable for training, we implement a verification pipeline that filters for both factual grounding and empirical retrieval difficulty. By confirming that a question is answerable given perfect information and remains solvable within a noisy, high-recall retrieval environment, we generate an intermediate-difficulty band that is challenging yet learnable. To address the bottlenecks imposed by sparse, outcome-only supervision, we pair this data with a retrieval-aware reward that explicitly incentivizes search quality and evidence selection rather than only final-answer correctness. Through extensive experiments, S^3 -R1 substantially improves over strong baselines on multiple multi-hop QA benchmarks.

2 Related Works

2.1 Large Language Models for Retrieval, Search, and Question Answering

While Large Language Models (LLMs) exhibit strong reasoning capabilities (Guo et al., 2025b), their reliance on static, parametric knowledge makes them susceptible to hallucinations and knowledge cutoffs (Zhang et al., 2023). Retrieval-Augmented Generation (RAG) mitigates this by grounding LLM outputs in external evidence (Lewis et al., 2020; Gao et al., 2023). However, the standard “retrieve-then-read” pipeline is brittle: retrieved context can include distracting or stale information (“context rot”) (Jin et al., 2024) and document ordering can be suboptimal (Pasupat et al., 2024). As a result, early work focused on improving retrieval quality, for example via query rewriting (Gao et al., 2024; Zhang et al., 2024; Karaki et al., 2024). A more powerful paradigm is active tool use, where the LLM acts as an agent that iteratively reasons, searches, and updates its evidence. Early approaches such as Chain-of-Thought (CoT) (Wei et al., 2022), ReAct (Yao et al., 2023), and IRCoT (Trivedi et al., 2022b) relied on carefully designed prompting frameworks for reasoning and tool-use, while supervised fine-tuning (SFT) requires expensive, large-scale labeled trajectories (Schick et al., 2023). However, for complex multi-hop questions, “semantic drift” (Xiong et al., 2022) from imprecise search queries during tool use often results in degraded performance. Therefore, more recent systems, such as Search-R1 and R1-Searcher, instead leverage reinforcement learning to train search-and-reason policies that decide what to query, how to use retrieved evidence, and when to stop to answer questions.

2.2 Reinforcement Learning for Large Language Models

While Proximal Policy Optimization (PPO) (Schulman et al., 2017) established the foundation for aligning large language models (Ouyang et al., 2022; Kaelbling et al., 1996; Sutton & Barto, 1999), the overhead induced by learning a value model has driven the search for simpler alternatives. Direct optimization strategies like DPO (Rafailov et al., 2023) and SimPO (Meng et al., 2024) bypass the reward model entirely, but frequently encounter off-policy degradation (Pang et al., 2024; Hsu et al., 2024). Conversely, recent on-policy methods focus on streamlining the RL pipeline itself. Algorithms such as GRPO (Shao et al., 2024), DAPO (Yu et al., 2025), RLOO (Ahmadian et al., 2024), and GSPO (Zheng et al., 2025) maintain training stability while completely discarding the value network. Because these critic-free frameworks excel at eliciting complex reasoning from sparse outcome rewards (Guo et al., 2025b), we utilize them in S^3 -R1 to post-train LLMs for multi-hop search following prior works (Jin et al., 2025b; Song et al., 2025).

2.3 Synthetic Data for Search and Tool Use

Training on synthetic data has shown to improve many LLMs (Nadas et al., 2025). Many synthetic data generation methods for reasoning, such as STaR (Zelikman et al., 2022), Rejection Finetuning (RFT) (Yuan et al., 2023), ReST (Gulcehre et al., 2023), and ReSTEM (Singh et al., 2023), rely on a ‘generate-and-filter’ approach. These techniques prompt a model to produce reasoning traces, such as chains-of-thought, and then perform Supervised Fine-Tuning (SFT) exclusively on the responses that lead to a correct final answer. In the retrieval and QA domain, LERET (Hsu et al., 2024) relies on generating synthetic instances of intermediate queries seeded from in-context examples are trained via DPO on outcomes. More recently, (Goldie et al., 2025) showed that learning from synthetic reasoning steps from a strong teacher can improve tool use in multi-turn settings. In contrast, our paper uses a stronger model to generate synthetic intermediate-difficulty questions from cases where the base agent is weak, enabling the weaker agent to learn from novel but easier problem instances.

3 Preliminaries

3.1 Multi-hop Search

Many methods, such as Search-R1 (Jin et al., 2025b) and R1-Searcher (Song et al., 2025) use a structured rollout for the LLM to search during QA. The generation of a complete response trajectory y , denoted as $y \sim \pi_\theta(\cdot|x, \mathcal{R})$ for a given prompt x and a search engine $\mathcal{R}$, is an interleaved sequence of text generation and tool calls, governed by a token-based protocol. The process begins with the LLM generating thoughts within the `<think>` and `</think>`. When external information is required, the model searches with tokens, `<search>` and `</search>`. The agent executes the search query through a search engine, and the retrieved results are enclosed within `<information>` and `</information>` tokens, and added to the context for further reasoning and generation. This trajectory is terminated when the model produces a final answer within `<answer>` and `</answer>` tokens or when a preset limit on search calls is reached.

3.2 Group Relative Policy Optimization for Multi-hop Search

Several RL methods have been introduced to post-train LLMs to optimize a reward function. Proximal Policy Optimization (PPO) (Schulman et al., 2017) optimizes the policy using a learned value function, whereas Group Relative Policy Optimization (GRPO), proposed by Shao et al. (2024), avoids the need for an explicit value function.

GRPO establishes a baseline for policy updates using the average reward from a group of sampled trajectories. For each input prompt x , GRPO (Shao et al., 2024) samples a group of G responses, $\{y_1, y_2, \dots, y_G\}$, from a policy π_{old} . The current policy, π_θ , is then optimized by maximizing the following objective function:

$$J_{\text{GRPO}}(\theta) = \mathbb{E}_{x \sim \mathcal{D}, \{y_i\}_{i=1}^G \sim \pi_{\text{old}}} \left[\frac{1}{G} \sum_{i=1}^G \frac{1}{\sum_{t=1}^{|y_i|} I(y_i, t)} \sum_{t=1, I(y_i, t)=1}^{|y_i|} \min \left(\frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t}|x, y_{i,<t})} \hat{A}_{i,t}, \right. \right. \\ \left. \left. \text{clip} \left(\frac{\pi_\theta(y_{i,t}|x, y_{i,<t})}{\pi_{\text{old}}(y_{i,t}|x, y_{i,<t})}, 1 - \epsilon, 1 + \epsilon \right) \hat{A}_{i,t} - \beta D_{\text{KL}}[\pi_\theta(y_{i,t}|x, y_{i,<t}) || \pi_{\text{ref}}(y_{i,t}|x, y_{i,<t})] \right) \right] \quad (1)$$

Here, ϵ and β are hyperparameters controlling the clipping ratio and the strength of the KL regularization, respectively, and π_{ref} is the reference policy. The advantage, $\hat{A}_{i,t} = \frac{r_i - \text{mean}(r)}{\text{std}(r)}$, where r denotes the per-sample reward computed within each group.

GRPO directly adds the KL divergence between the trained policy π_θ and the reference policy π_{ref} to the loss function as a regularization term. For search, a token mask $I(y_i, t)$ is applied to mask the information obtained from external sources (Jin et al., 2025b).

4 Method

This section outlines our training methodology for S^3 -R1. To successfully train models for complex, multi-hop reasoning, we pivot away from relying solely on standard optimization tweaks and instead emphasize a data-centric approach coupled with dense reward signals. We first broaden the training distribution via a rigorous synthetic question generation pipeline that mines hard seeds, generates diverse questions, and filters them based on retrieval-solvability. We then pair this enhanced data distribution with a retrieval-aware reward that augments outcome-only supervision with a recall-based signal to directly reward search quality, alongside final answer correctness.

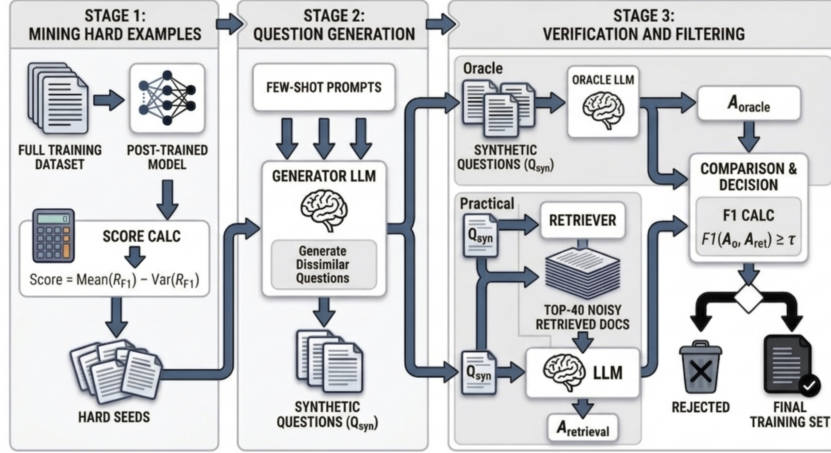


Figure 1: **Synthetic Data Generation Pipeline.** We mine hard anchor questions by scoring training prompts with a risk-adjusted solvability metric (mean minus variance over 5 rollouts) and selecting the lowest-scoring 10K instances. Conditioned on each anchor’s evidence documents, a generator model (Gemini 2.5 Pro) produces dissimilar synthetic questions. We then verify solvability under retrieval by comparing an oracle answer from the ground-truth documents to an answer produced from BM25 top-40 retrieved documents, retaining questions whose token-level F1 exceeds a threshold.

4.1 Synthetic Data Generation for Enhanced Training

To broaden the coverage of extended reasoning and rebalance the training distribution, we augment our training set with synthetically generated questions via a pipeline comprising three distinct phases: hard example mining (isolating hard questions), question generation (converting hard questions to synthetic questions), and retrieval-based verification (finding questions of intermediate difficulty with their answers).

4.1.1 Identifying Hard Anchor Instances

We synthesize questions by first identifying hard anchor questions that are typically characterized by lower average accuracies and high stochasticity from the model. We begin by post-training the model on the entire existing dataset $\mathcal{Q}$. For each prompt $q \in \mathcal{Q}$, we evaluate its empirical difficulty using the best-performing checkpoint by sampling $K = 5$ independent reasoning trajectories. We then calculate a pessimistic (La & Ghavamzadeh, 2013) solvability score $Score(q)$ based on the mean $\mathbb{E}[R_{F1}]$ and sample variance $\text{Var}[R_{F1}]$ of the F1 scores as

$$Score(q) = \mathbb{E}_{a \sim \pi_\theta} [R_{F1}(q, a)] - \text{Var}_{a \sim \pi_\theta} [R_{F1}(q, a)].$$

Because the F1 score provides a continuous evaluation signal, incorporating the variance penalty per group establishes a lower confidence score (Buckman et al., 2020) that explicitly heavily penalizes questions exhibiting high epistemic uncertainty. This fundamentally reorders questions, ensuring that unstable, high-variance instances are driven further down

the lower tail alongside instances of consistent failure. Consequently, this scoring mechanism isolates the actual boundary of the model’s competence, implicitly identifying multi-hop questions as hard anchor examples. We then select the 10,000 questions with the lowest $Score(q)$ values as anchor examples for synthetic question generation.

4.1.2 Dissimilarity-Driven Question Generation

In the second phase, we utilize a few-shot prompting strategy with a highly capable generator model (Gemini 2.5 Pro) to synthesize high-quality, relevant questions. We construct a context-rich prompt comprising several in-context examples, where each exemplar pairs a set of source documents with a corresponding high-quality question sampled from our pool of mined anchor instances. Then, to this context, we append the target document set along with its original anchor question to generate a new synthetic question grounded in the target document set. Crucially, we instruct the generator to synthesize questions that diverge from the original anchor question and further filter out highly similar questions to the original anchor question. Moreover, by systematically randomizing the in-context exemplars, the generator model generates a more varied distribution of new problems. The exact prompts utilized for this generation process are detailed in the Appendix.

4.1.3 Verification and Filtering

A generated question is only viable for training if it is both factually solvable from the source material and practically retrievable given the limitations of the standard retrieval tool utilized. To ensure these conditions are met, we introduce a two-phase verification process for the synthetic questions. First, we establish an upper bound on factual solvability by prompting the generator model (Gemini 2.5 Pro) alongside the ground-truth documents to produce an oracle answer, A_{oracle} . Second, we evaluate the empirical retrieval difficulty. Because the downstream model interacts with the corpus using a relatively weak lexical retriever (BM25), questions that demand excessively complex semantic matching to surface the relevant text are practically unsolvable during training. To filter out these intractable instances, we fetch the top- k documents via BM25 and tasking the generator to produce a retrieval-based answer, $A_{\text{retrieval}}$, relying strictly on this retrieved context.

We set $k = 40$ to create a high-recall, low-precision environment; if the generator can extract the necessary evidence from this noisy context, it confirms that the weak retriever is at least capable of surfacing the required information. During actual RL training, the agent is heavily constrained, permitted to retrieve only 5 documents per turn across a maximum of 5 turns. Consequently, questions that pass this 40-document verification step represent an optimal challenge: they are difficult enough to incentivize the model to learn highly precise, iterative search strategies, yet grounded enough to be solvable without requiring a prohibitively powerful semantic retriever.

Finally, we evaluate the agreement between the two answers by calculating a token-level F1 score, retaining only questions where $F1(A_{\text{oracle}}, A_{\text{retrieval}}) \geq \tau$, with τ serving as the acceptance threshold. We prioritize F1 over EM to prevent aggressively discarding questions. This verification pipeline yields a filtered synthetic dataset that is challenging, factually grounded, and reliably solvable under the constraints of a multi-hop retrieval environment.

4.2 Reward Formulation

Standard reward functions for question-answering are typically sparse, relying entirely on a binary Exact Match (EM) signal at the end of a trajectory. For multi-hop QA, this sparse formulation suffers from severe credit assignment issues; a model might execute three perfect search queries but fail on the final synthesis, receiving a zero reward that inadvertently penalizes its excellent search behavior. To provide a denser, step-aware learning signal, we formulate a composite reward that evaluates both the final correctness and the intermediate quality of the information retrieval process. Here, R averages the Exact Match score and the Cumulative Recall of the retrieved documents:

$$R = \frac{R_{\text{EM}} + R_{\text{Recall}}}{2}.$$

Here, $R_{EM} \in \{0, 1\}$ is the binary correctness of the final answer, and R_{Recall} calculates the fraction of ground-truth documents successfully retrieved across all search turns in the trajectory. By rewarding intermediate retrieval, the policy receives a positive learning signal for searching documents even if the final answer is incorrect.

4.3 Stabilization of Reinforcement Learning Training

Because standard GRPO can be susceptible to premature convergence and erratic updates over long multi-hop rollouts, we integrate three established stabilization techniques to ensure robust learning. First, following DAPO (Yu et al., 2025), we apply double clipping with an elevated upper bound to preserve learning signals for low-probability exploration tokens. Concurrently, we increase the entropy loss coefficient (Schulman et al., 2017) to actively prevent the policy from collapsing into suboptimal, deterministic search patterns early in training. Finally, to address gradient instability caused by negative advantages (Jin et al., 2025b), we follow Ye et al. (2020) by applying a strict lower bound to the importance ratio $r_t(\theta)$. Specifically, when the advantage $\hat{A}_t < 0$, we cap the penalty magnitude by lower-bounding the standard GRPO objective with $\epsilon_{neg} \hat{A}_t$ (where $\epsilon_{neg} > 1$).

5 Experiments

Table 1: Pass@8 Performance Comparison on Multi-Hop QA Datasets

Methods	CofCA		Musique		2Wiki		HotpotQA	
	EM	Recall	EM	Recall	EM	Recall	EM	Recall
Qwen2.5-7b Instruct								
Direct Inference	0.288	-	0.039	-	0.378	-	0.220	-
RAG	0.283	53.2	0.028	43.5	0.205	62.3	0.228	72.5
CoT + RAG	0.403	53.2	0.060	43.5	0.299	62.3	0.328	72.5
Multihop + RAG	0.335	53.2	0.098	61.1	0.400	83.0	0.354	80.7
Search-R1	0.401	52.7	0.408	69.9	0.586	90.4	0.513	81.3
S³-R1- No Synthetic	0.521	54.0	0.448	69.9	0.663	88.2	0.576	83.3
S³-R1	0.605	52.9	0.460	69.1	0.694	90.3	0.599	85.5
Gemini 2.5 Pro								
RAG	0.418	43.3	0.171	50.9	0.325	66.9	0.352	63.8
COT + RAG	0.514	55.2	0.144	51.1	0.297	67.5	0.385	78.7
Decomp + RAG	0.522	55.3	0.199	62.6	0.329	70.7	0.430	83.0
MultiHop + RAG	0.573	55.7	0.452	81.5	0.651	87.8	0.587	85.1

We conduct a series of experiments to comprehensively evaluate the effectiveness of our proposed methodology, which combines RL post-training enhancements with a synthetic data pipeline. Our evaluation is designed to answer the following key research questions:

1. **Impact of RL Enhancements:** To what extent do the RL post-training enhancements improve performance over the baseline Search-R1?
2. **Impact of Synthetic Data:** How does augmenting the training data with our synthetic, hard examples affect both in-domain and out-of-domain performance?
3. **Ablation of Synthetic Data:** What is the overall contribution of the different components of our synthetic data generation pipeline?

Datasets and Metrics. We evaluate our models on four challenging multi-hop question-answering benchmarks: Musique (Trivedi et al., 2022a), HotpotQA (Yang et al., 2018), 2WikiMultiHopQA (Ho et al., 2020), and CofCA (Wu et al., 2024a). All models are trained exclusively on the Musique training set. We evaluate performance using two primary metrics. **Exact Match (EM)** measures the fraction of model-generated answers that exactly

match the ground truth. **Recall** calculates the fraction of necessary ground-truth documents that were successfully retrieved during the model’s search steps.

Table 2: Pass@1 (8 sample average) Performance Comparison on Multi-Hop QA Datasets.

Methods	Cofca		Musique		2Wiki		HotpotQA	
	EM	Recall	EM	Recall	EM	Recall	EM	Recall
Qwen2.5-7b Instruct								
Direct Inference	0.057	-	0.007	-	0.091	-	0.052	-
RAG	0.047	53.2	0.005	43.5	0.040	62.3	0.044	72.5
CoT + RAG	0.092	53.2	0.011	43.5	0.068	62.3	0.082	72.5
Multihop + RAG	0.068	29.2	0.015	24.3	0.083	41.1	0.079	36.5
Search-R1	0.184	38.2	0.254	53.8	0.338	73.1	0.298	56.3
S³-R1- No Synthetic	0.358	46.4	0.317	60.5	0.475	77.7	0.441	72.1
S³-R1	0.455	49.7	0.345	61.9	0.519	83.8	0.483	78.1

Baselines and Models. Our primary experiments are conducted using the open-source **Qwen2.5-7B Instruct** model for its strong instruction-following (Jin et al., 2025b). We compare against a spectrum of baselines to provide a thorough analysis:

- **Direct:** The model is asked to answer the question based on its internal knowledge.
- **RAG Methods:** Standard Retrieval-Augmented Generation (RAG) with context obtained through a BM25 retriever; Chain-of-Thought with RAG, where the model is asked to think carefully step by step to answer questions with the provided context (CoT + RAG); Multi-hop RAG baseline similar to the setup in 3.1.
- **Trained Models:** We post-train Qwen2.5-7b Instruct with Search-R1 and S³-R1.

We also evaluate **S³-R1** against a suite of advanced RAG prompting strategies with **Gemini 2.5 Pro**, including standard RAG, CoT + RAG, a decomposition-based approach (Decomp + RAG), which first decomposes the original question to sub-questions and retrieves documents for each sub-question before answering the original question, and Multi-Hop RAG.

Experimental Setup. All models are trained solely on the Musique dataset to rigorously assess zero-shot generalization performance on HotpotQA, 2WikiMultiHopQA, and CofCA. We use a consistent retriever (BM25) to ensure fair comparisons. While stronger retrievers would improve results Jin et al. (2025a), we expect this to improve all methods proportionally. Further details regarding hyperparameters, the training process, and our experimental infrastructure can be found in the Appendix.

Exact Match vs. Training Step on Musique

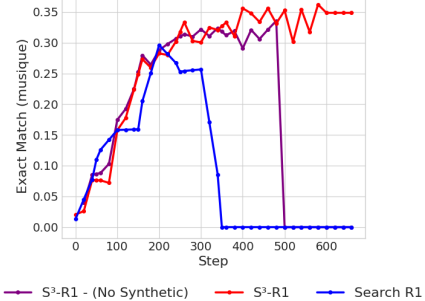


Figure 2: **Impact of RL algorithm changes on training.** We show that post-training Qwen2.5-7B without RL enhancements (purple) is more stable than Search-R1 (blue).

6 Discussions

6.1 Impact of our RL Algorithm Enhancements

To isolate the effect of our proposed RL algorithm enhancements, we first compare the performance of our model (**S³-R1 - No Synthetic**) against the baseline **Search-R1**. As shown in Table 1, our method demonstrates a notable improvement in both Exact Match and Recall

across all datasets. This indicates that the algorithmic modifications, including negative advantage clipping and the denser reward function, stabilize training.

Furthermore, we analyze the training dynamics of both models. Figure 2 plots the reward over the course of training and reveals a critical difference in stability. While the baseline Search-R1 is prone to performance collapses and high variance, our enhanced algorithm maintains a stable and monotonically increasing reward curve for a much longer training duration. This stability is particularly beneficial for complex queries that require longer reasoning chains or multiple search "hops." By mitigating erratic policy updates, the model can learn to make longer learning progress on harder problems, leading to superior performance on the most challenging questions.

6.2 Impact of Synthetic Data

We assess the contribution of our synthetic data pipeline by comparing the full model (S^3 -R1) to a version trained only on the original data. The results in Table 1, Table 2, and Table 3 show that the inclusion of synthetic data yields a significant additional performance boost. Importantly, these improvements extend beyond the in-domain Musique benchmark to out-of-domain datasets, indicating that our augmentation enhances generalization rather than overfitting to a single corpus.

The training curves in Figure 2 further corroborate the influence of the synthetic questions. The model trained on the mixture of synthetic and original questions consistently achieves a higher average pass@1 performance averaged over 8 passes. We believe that by injecting synthetic questions of intermediate difficulty, the model develops transferable multi-hop reasoning ability.

Table 3: Per-Hop Exact Match (EM) Pass@8 Performance on Cofca and Musique Datasets

Methods	Cofca Exact Match Score				Musique Exact Match Score			
	2-Hop	3-Hop	4-Hop	Overall EM	2-Hop	3-Hop	4-Hop	Overall EM
Qwen2.5-7b Instruct								
Multihop + RAG	0.275	0.432	0.300	0.335	0.143	0.060	0.033	0.098
Search-R1	0.406	0.329	0.462	0.401	0.501	0.347	0.231	0.408
S^3 -R1- No Synthetic	0.437	0.558	0.570	0.521	0.538	0.361	0.337	0.448
S^3 -R1	0.482	0.653	0.679	0.605	0.526	0.409	0.351	0.460
Gemini 2.5 Pro								
RAG	0.413	0.457	0.384	0.418	0.218	0.136	0.088	0.171
COT + RAG	0.390	0.558	0.592	0.514	0.198	0.100	0.049	0.144
Decomp + RAG	0.390	0.555	0.618	0.522	0.255	0.160	0.097	0.199
MultiHop + RAG	0.460	0.672	0.589	0.573	0.529	0.426	0.262	0.452

6.3 Ablation on Synthetic Data Generation

The effectiveness of our synthetic data pipeline depends on two key hypotheses: (1) that seeding from "hard" examples is crucial, and (2) that a rigorous verification process is necessary to ensure data quality. To validate these design choices, we conduct an ablation study with two alternative training runs:

- **Random Verified:** A model trained with synthetic data generated from randomly selected, easier examples instead of the hard-mined seeds.
- **Hard Unverified:** A model trained with synthetic data generated from hard-mined seeds but without the final verification and filtering step.
- **Only Synthetic, Hard Verified:** A model trained only on synthetic data generated from hard-mined seeds with verification.

Figure 3 presents the average pass@1 performance over recall and exact match for these ablations compared to our full method. The results clearly demonstrate that both components

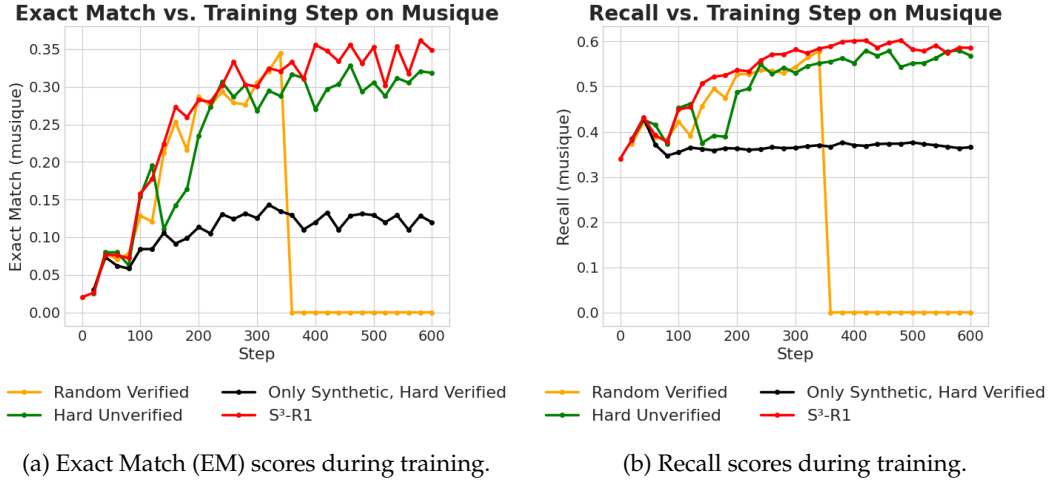


Figure 3: **Ablation of synthetic data generation components on training.** The left figure shows the Exact Match performance, while the right shows Recall. Our model trained with our RL enhancement on a mixture of original and synthetic data obtained from our proposed pipeline (Red) outperforms all other variants for compiling synthetic data on Pass@1 performance.

are critical. The model trained on unverified data performs worse, indicating that the verification step is essential for filtering out noisy or unsolvable questions that would otherwise introduce misleading signals into the training process. This is further corroborated by the performance of the model on only synthetic data, where we observe no learning progress. Moreover, when we replace mined hard seeds with randomly initialized seeds, training becomes unstable after roughly 300 steps, and therefore shows a diminished performance gain. These results confirm that our hard-example mining strategy successfully targets the model’s weaknesses and drives more meaningful learning progress.

7 Conclusion

In this work, we tackled two key bottlenecks in training LLMs for multi-hop search and QA, limited coverage of complex training questions, and sparse, outcome-only rewards. S³-R1 combines (i) a synthetic generation-and-curation pipeline that expands the training distribution with verified, intermediate-hardness multi-hop questions, and (ii) a retrieval-aware, denser RL learning signal that rewards not only final-answer correctness but also intermediate search quality and evidence selection. Together with stabilization techniques that prevent policy collapse during long-horizon optimization, these components yield more effective search-and-synthesis policies and robust generalization across both in-domain and out-of-domain evaluations on multi-hop QA datasets.

8 Future Work

Building on these results, a key direction is to further strengthen both the data pipeline and the supervision used for long-horizon search. On the data side, we will explore using stronger generative models to decompose complex multi-hop questions into sequences of grounded sub-questions, enabling a more structured curriculum and finer-grained control over question hardness during verification. On the optimization side, we plan to enrich our retrieval-aware reward with explicit process supervision, providing step-level feedback on query formulation, evidence selection, and stopping decisions. We expect that jointly scaling hardness-aware curricula and process-level learning signals will be central to training more robust long-horizon search agents.

References

- Arash Ahmadian et al. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. *arXiv preprint arXiv:2402.14740*, 2024.
- Anthropic. Claude 3. 2024. URL <https://www.anthropic.com/news/claude-3-family>.
- Jacob Buckman, Carles Gelada, and Marc G Bellemare. The importance of pessimism in fixed-dataset policy optimization. *arXiv preprint arXiv:2009.06799*, 2020.
- Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. Self-questioning language models. *arXiv preprint arXiv:2508.03682*, 2025.
- Liang Gao et al. In-context learning for query rewriting. *arXiv preprint arXiv:2502.15009*, 2024.
- Yunfan Gao et al. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.
- Anna Goldie, Azalia Mirhoseini, Hao Zhou, Irene Cai, and Christopher D. Manning. Synthetic data generation & multi-step rl for reasoning & tool use. *arXiv preprint arXiv:2504.04736*, 2025.
- Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (rest) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.
- Daya Guo et al. Deepseek-r1: Pushing the limits of reasoning with reinforcement learning. 2025a.
- Daya Guo et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025b.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. *arXiv preprint arXiv:2011.01060*, 2020.
- Sheryl Hsu, Omar Khattab, Chelsea Finn, and Archit Sharma. Grounding by trying: Llms with reinforcement learning-enhanced retrieval. *arXiv preprint arXiv:2410.23214*, 2024.
- Bowen Jin, Jinsung Yoon, Jiawei Han, and Serkan O Arik. Long-context llms meet rag: Overcoming challenges for long inputs in rag. *The Thirteenth International Conference on Learning Representations*, 2024.
- Bowen Jin, Jinsung Yoon, Priyanka Kargupta, Serkan O Arik, and Jiawei Han. An empirical study on reinforcement learning for reasoning-search interleaved llm agents. *arXiv preprint arXiv:2505.15117*, 2025a.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan O. Arik, Dong Wang, Hamed Zamani, and Jiawei Han. Search-r1: Training llms to reason and leverage search engines with reinforcement learning. *arXiv preprint arXiv:2503.09516*, 2025b.
- Leslie Pack Kaelbling, Michael L Littman, and Andrew W Moore. Reinforcement learning: A survey. *Journal of artificial intelligence research*, 4:237–285, 1996.
- Fares Karaki et al. Prewrite search: A reinforcement learning approach to query rewriting. *arXiv preprint arXiv:2401.08189*, 2024.
- Prashanth La and Mohammad Ghavamzadeh. Actor-critic algorithms for risk-sensitive mdps. *Advances in neural information processing systems*, 26, 2013.
- Patrick Lewis et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.

- Yu Meng, Mengzhou Xia, and Danqi Chen. Simpo: Simple preference optimization with a reference-free reward. *Advances in Neural Information Processing Systems*, 37, 2024.
- Mihai Nadas, Laura Diosan, and Andreea Tomescu. Synthetic data generation using large language models: Advances in text and code. *arXiv preprint arXiv:2503.14023*, 2025.
- OpenAI. Gpt-4o. 2024. URL <https://openai.com/index/hello-gpt-4o/>.
- Long Ouyang et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- Richard Yuanzhe Pang et al. Iterative reasoning preference optimization. *Advances in Neural Information Processing Systems*, 37, 2024.
- Panupong Pasupat et al. Ensemble of llm-retriever for accurate document ranking. In *arXiv preprint arXiv:2501.00332*, 2024.
- Rafael Rafailov et al. Direct preference optimization: Your language model is secretly a reward model. *Advances in Neural Information Processing Systems*, 36, 2023.
- Timo Schick et al. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Avi Singh, John D Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *arXiv preprint arXiv:2312.06585*, 2023.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Ji-Rong Wen. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*, 2025.
- Richard S Sutton and Andrew G Barto. Reinforcement learning. *Journal of Cognitive Neuroscience*, 11(1):126–134, 1999.
- Gemini Team et al. Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. *arXiv preprint arXiv:2403.05530*, 2024.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554, 2022a.
- Harsh Trivedi et al. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. *arXiv preprint arXiv:2212.10509*, 2022b.
- Jason Wei et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- Jian Wu, Linyi Yang, Zhen Wang, Manabu Okumura, and Yue Zhang. Cofca: A step-wise counterfactual multi-hop qa benchmark. *arXiv preprint arXiv:2402.11924*, 2024a.
- Zixiang Wu, Yang Fan, Zhiyong Wu, et al. Cofca: A comprehensive and challenging benchmark for tool-assisted reasoning. *arXiv preprint arXiv:2402.12212*, 2024b.
- Wenhan Xiong et al. Iterative multi-hop retrieval. *arXiv preprint arXiv:2204.09140v2*, 2022.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. *arXiv preprint arXiv:1809.09600*, 2018.

- Shunyu Yao et al. React: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- Deheng Ye, Zhao Liu, Mingfei Sun, Bei Shi, Peilin Zhao, Hao Wu, Hongsheng Yu, Shaojie Yang, Xipeng Wu, Qingwei Guo, et al. Mastering complex control in moba games with deep reinforcement learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 6672–6679, 2020.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, et al. Dapo: An open-source llm reinforcement learning system at scale. *arXiv preprint arXiv:2503.14476*, 2025.
- Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models, 2023. URL <https://arxiv.org/abs/2308.01825>.
- Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah Goodman. Star: Bootstrapping reasoning with reasoning. *Advances in Neural Information Processing Systems*, 35:15476–15488, 2022.
- Zhiyuan Zeng, Hamish Ivison, Yiping Wang, Lifan Yuan, Shuyue Stella Li, Zhuorui Ye, Siting Li, Jacqueline He, Runlong Zhou, Tong Chen, et al. RLve: Scaling up reinforcement learning for language models with adaptive verifiable environments. *arXiv preprint arXiv:2511.07317*, 2025.
- Yue Zhang et al. Siren’s song in the ai ocean: a survey on hallucination in large language models. *arXiv preprint arXiv:2309.01219*, 2023.
- Yuxin Zhang et al. Maerfw: A curriculum-based reinforcement learning framework. *arXiv preprint arXiv:2408.17072*, 2024.
- Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

A Appendix

A.1 Training Details

Datasets. We train exclusively on the MuSiQue training split, and evaluate using Exact Match (EM) on the test/validation sets of MuSiQue (in-domain) and CoFCA, 2Wiki-MultiHopQA, and HotpotQA (out-of-domain). Unlike Search-R1, which trains on a HotpotQA+NQ mixture, we deliberately use MuSiQue due to its smaller size (supporting faster, reproducible iterations) and its diverse composition of multi-step reasoning questions that align with our multi-hop objectives.

Synthetic data mixture. To rebalance toward harder, multi-hop queries, we augment the training pool with verified synthetic questions produced by our pipeline (hard-seed mining, few-shot question generation with dissimilarity constraints, and oracle-vs-retrieval verification). Augmented models sample from the union of original and synthetic items during RL rollouts; unaugmented baselines use only the original corpus. Unless otherwise stated, retrieval and optimization settings are identical across augmented and unaugmented runs.

Models and retrieval. All experiments use the Qwen-2.5-7B Instruct model. We choose this model for experiments as they were shown to be the most performant in [Jin et al. \(2025b\)](#). For retrieval, we index the entire corpus of documents referenced within the Musique dataset with the BM25 retriever. Each retrieval-based method consumes the top-5 passages per query.

Table 4: Training and rollout hyperparameters. Common settings apply to both Search-R1 and S³-R1 unless noted.

Hyperparameter	Search-R1	S ³ -R1
Training steps	600	600
Policy LLM learning rate	1e-6	1e-6
Warm-up ratio (policy)	0.285	0.285
Samples per prompt (rollouts)	5	5
KL regularization coefficient β	0.001	0.001
Entropy Coefficient λ	0.005	0.005
Clip ratio Low ϵ_{low}	0.2	0.2
Clip ratio High ϵ_{high}	0.2	0.32
Negative Advantage Clip ϵ_{neg}	—	3.0
Total / mini / micro batch size	256 / 128 / 32	256 / 128 / 32
Max input sequence length (tokens)	8192	8192
Max response length (tokens)	500	500
Max retrieved content (tokens)	2048	2048
Action budget B (search steps)	5	5
Retriever top- k (passages)	5	5
Rollout temperature / top- p	1.0 / 1.0	1.0 / 1.0
vLLM tensor-parallel / mem util.	1 / 0.6	1 / 0.6
Checkpoint save frequency (steps)	100	100
Divergence handling	Evaluate most recent stable checkpoint	
Compute and parallelism	Single node, 8×H100; FSDP with CPU offload	

A.2 Prompts

This section details the prompts used for synthetic data generation, verification, and baseline evaluations.

A.2.1 Synthetic Data Generation and Verification

Question Generation. This prompt is used to generate new, complex questions from a given set of documents.

Question Generation Prompt

You are an expert at creating high-quality questions from a given text corpus. Your task is to generate a single, new, and insightful question that can be answered **only** by using the information present in the provided documents. The new question should be different from the original question provided. The question should require synthesizing information, not just simple fact retrieval.

Now, based on the following documents, generate a new question. The new question must be different from the original question. First think carefully about the provided documents, and then generate a question based on them. Make sure you generate questions that would require all the documents to answer and not just a simple fact retrieval from the documents. Provide only the question itself, without any preamble.

Answer Verification. This prompt is used to verify if a generated question can be answered from a set of ground-truth documents, ensuring data quality.

Answer Verification Prompt

You are a Question Answering system. Your task is to answer the following question based **only** on the information present in the provided documents. Do not use any external knowledge. Explicitly think about the question. Not all documents might be relevant to the question, hence you must carefully analyze which documents would be useful to answering your question. Finally provide an answer once you are reasonably confident of the answer.

****META INSTRUCTION:**** Provide the most concise or shortest answer as possible that is no longer than 5 words. If the answer cannot be found in the documents, respond with 'Answer not found in documents.'

— DOCUMENTS — documents
— QUESTION — question
— ANSWER —

A.2.2 Baseline Evaluation Prompts

Standard RAG. This prompt instructs the model to answer a question based only on a provided context, with a meta-instruction for conciseness.

Standard RAG Prompt

You are a Question Answering system. Your task is to answer the following question based **only** on the information present in the provided documents. Do not use any external knowledge. Explicitly think and reason about the question step by step. Not all documents might be relevant to the question, hence you must carefully analyze which documents would be useful to answering your question. Finally provide an answer once you are reasonably confident of the answer.

****META INSTRUCTION:**** Provide the most concise or shortest answer as possible that is no longer than 5 words. If the answer cannot be found in the documents, respond with 'Answer not found in documents'.

— Information — search_results
— Question — question
Answer:

Decomposition-based RAG. This method uses two prompts: one to decompose the question and another to synthesize the final answer.

Decomposition Prompt

You are an advanced AI designed to function as a sophisticated query decomposer. Your primary objective is to meticulously analyze complex, multi-faceted questions and systematically break them down into a series of simpler, discrete, and self-contained sub-questions. Each sub-question should be formulated in such a way that it can be independently and effectively answered by a standard search engine, facilitating a step-by-step approach to resolving the original complex query. Your output must adhere to a strict JSON format. Specifically, you are required to return the generated sub-questions as a JSON list of strings. This JSON list should be encapsulated within a JSON markdown block for clear presentation and easy parsing.

For illustrative purposes, consider the following example: **Question:** What is the capital of the southernmost country in the continent with the longest river in the world? **Sub-questions:**

```
[
  "What is the longest river in the world?",
  "Which continent has the longest river in the world?",
  "What is the southernmost country of that continent?",
  "What is the capital of that country?"
]
```

Please provide the sub-questions for the following complex query: **Question:** question **Sub-questions:**

Synthesis Prompt

You are an advanced Question Answering system designed to provide precise and concise answers. Your primary objective is to address the **Original Question** by leveraging information exclusively from the **Documents** and the insights gained from answering the **Sub-questions**. The **Sub-questions** are meticulously crafted, discrete, and self-contained inquiries derived from the **Original Question**, serving as a structured approach to facilitate a comprehensive answer. You must thoroughly analyze the **Original Question** and each **Sub-questions**, then synthesize the relevant information from the **Documents** to formulate your final response. It is imperative that you strategically utilize the **Sub-questions** to explicitly think step by step and guide your information extraction and synthesis from the **Documents**. Under no circumstances should you incorporate any external knowledge beyond the provided **Documents**.

META INSTRUCTION: Your answer must be the most concise possible, limited to a maximum of 5 words. If the definitive answer cannot be explicitly found in the provided **Documents**, you must respond with the exact phrase 'Answer not found in documents'.

— Original Question — question
 — Sub-questions — sub.questions.formatted
 — Documents — aggregated.context
 Answer:

Iterative Multi-Hop RAG. This prompt guides the model to perform a sequence of search-and-think steps, one hop at a time.

Multi-Hop RAG Prompt

You are a helpful assistant who will answer the given question step by step. First, think about the question in `<think></think>` tags. Then, if needed, create a search query in `<search></search>` tags. Information will be provided to you in `<information></information>` tags. You can perform multiple searches. When you have enough information, provide the final answer between `<answer></answer>` tags.

Here is a correct example of a sample response every hop. Note that you are only allowed to think, search and answer once per hop. Information is provided to you based on the search query through an external tool.

****Question:**** What is the capital of the southernmost country of the continent with the longest river in the world? ****Hop 1:**** `<think> I need to know the longest river </think> <search> longest river </search>` ****Hop 2:**** `<think> From information Nile is the longest river, I need to know continent </think> <search> continent where Nile is located </search>` ****Hop 3:**** `<think> From information, Nile is located in Africa, let's find the southernmost country in Africa </think> <search> southernmost country in Africa </search>` ****Hop 4:**** `<think> southernmost country of Africa is South Africa, let's search for its capital </think> <search> capital of South Africa </search>` ****Hop 5:**** `<think> Based on the docs it's Pretoria </think> <answer> Pretoria </answer>`

****Bad examples:**** Generating your own information per hop is not permitted. Hop 1: `<think> I need to know the longest river </think> <search> longest river </search> <information> ... </information>`

****Bad examples:**** Generating multiple hops in a single response is not permitted. Hop 1: `<think> I need to know the longest river </think> <search> longest river </search>` Hop 2: `<think> From information Nile is the longest river, I need to know continent </think> <search> continent where Nile is located </search>`

****META INSTRUCTION:**** Provide the most concise or shortest answer as possible that is no longer than 5 words. If the answer cannot be found in the documents, respond with 'Answer not found in documents'.

You must only search, think or answer once every hop. You must not generate or use the `<information>` and `</information>` to hallucinate information, it will be provided to you. Do not use any external knowledge when drafting the query, thinking step and answer. Use information you have gathered through search queries.

****Question:**** question

A.2.3 RL training Prompt

For reinforcement learning, the model is guided by the following instruction-based prompt. This prompt encourages the generation of explicit reasoning and search actions, which form the basis of the policy’s action space.

RL Training Prompt

You are a helpful assistant who will answer the given question with some potentially useful context step by step. You should analyze the question carefully, evaluate the given context (which may or may not be useful), and then generate an accurate and well-reasoned response. You should first have a reasoning process in mind. You must show your reasoning in `<think>` `</think>` tags. After reasoning, if you find you lack some knowledge, you create a query for a search engine enclosed within the `<search>` `</search>` tags and it will provide you the necessary information between the `<information>` and `</information>`. You must not generate or hallucinate any information in the `<information>` and `</information>` tags, it will be provided to you. You can search as many times as you need. If you think you have sufficient information to answer, you must return final answer in `<answer>` `</answer>` tags. Here are some examples for you to follow, note that the information is provided from the search engine, and you can ask for more information if you need. For example: Question: What is the capital of China? `<think>` I must search the capital of the China `</think>` `<search>` Capital of China `</search>` `<information>` Beijing is the capital of China `</information>` `<think>` Based on the information its Beijing `</think>` `<answer>` Beijing `</answer>`.
 Question: Is Beijing the capital of China? `<think>` I must search the capital of the China `</think>` `<search>` Capital of China `</search>` `<information>` Beijing is the capital of China `</information>` `<think>` Based on the information its Beijing `</think>` `<answer>` Yes `</answer>`.
 Question: How many capitals did China have? `<think>` The user is asking for capitals of China in the present and past `</think>` `<search>` List current capital of China `</search>` `<information>` Beijing is the capital of China `</information>` `<think>` I only know the present capital from the given information, I need to search for any capitals of China in the past `</think>` `<search>` List past capitals of China `</search>` `<information>` There are traditionally four major historical capitals of China referred to as the "Four Great Ancient Capitals of China". The four are Beijing, Nanjing, Luoyang and Xi'an (Chang'an). `</information>` `<think>` Based on the information its 4 `</think>` `<answer>` 4 `</answer>`.
 Question: question

B LLM Usage Declaration

We used Gemini 2.5 Pro to improve the grammar and flow of the final draft. We also used LLMs to paraphrase and condense the verification section in the Methods, based on a human-written version. Additionally, we used the Overleaf AI Assistant to address formatting issues in the paper’s tables and figures. Figure 1 was generated with assistance from Nano Banana Pro based on a flowchart created by the authors on slides.